Skill-3

```
==================================
 COMMAND HISTORY & DYNAMIC BUFFER
==================================

Escape Sequences:
New Line      : \n
Tab           : \tExample
Backslash     : \
Double Quote  : "Hello"

Enter commands.
Type 'history' to display history.
Type 'prev' for previous command.
Type 'next' for next command.
Type 'exit' to quit.

Command: ls
Command stored: ls
Command: pwd
Command stored: pwd
Command: gcc program.c
Buffer resized from 10 to 20 bytes.
Command stored: gcc program.c
Command: prev
Previous command: gcc program.c
Command: prev
Previous command: pwd
Command: next
Next command: gcc program.c
Command:
```

```
himasree@LAPTOP-41COT5AA:~/OSLab$ cat histoiry.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define INITIAL_SIZE 10

/* ---------------- COMMAND HISTORY ---------------- */

typedef struct Node {
    char *command;
    struct Node *next;
    struct Node *prev;
} Node;

Node *head = NULL;
Node *tail = NULL;
Node *current = NULL;

/* Add command to history */
void addCommand(const char *cmd) {
    Node *newNode = malloc(sizeof(Node));

    if (newNode == NULL) {
        printf("Memory allocation failed!\n");
        return;
    }

    newNode->command = malloc(strlen(cmd) + 1);
```

```
    if (newNode->command == NULL) {
        free(newNode);
        printf("Memory allocation failed!\n");
        return;
    }

    strcpy(newNode->command, cmd);

    newNode->next = NULL;
    newNode->prev = tail;

    if (tail != NULL)
        tail->next = newNode;
    else
        head = newNode;

    tail = newNode;
}

/* Show previous command */
void previousCommand() {
    if (current == NULL)
        current = tail;
    else if (current->prev != NULL)
        current = current->prev;

    if (current != NULL)
        printf("Previous command: %s\n", current->command);
    else
        printf("No previous command.\n");
```

```
}

/* Show next command */
void nextCommand() {
    if (current == NULL) {
        printf("No next command.\n");
        return;
    }

    if (current->next != NULL) {
        current = current->next;
        printf("Next command: %s\n", current->command);
    } else {
        printf("Already at latest command.\n");
    }
}

/* Display complete history */
void showHistory() {
    Node *temp = head;

    printf("\nCommand History:\n");

    while (temp != NULL) {
        printf("%s\n", temp->command);
        temp = temp->next;
    }
}
```

```
void freeHistory() {
    Node *temp;

    while (head != NULL) {
        temp = head;
        head = head->next;

        free(temp->command);
        free(temp);
    }

    tail = NULL;
    current = NULL;
}

/* ---------------- DYNAMIC BUFFER ---------------- */

char *createBuffer(int size) {
    char *buffer = malloc(size);

    if (buffer == NULL) {
        printf("Memory allocation failed!\n");
        exit(1);
    }

    return buffer;
}

/* Resize buffer safely */
char *resizeBuffer(char *buffer, int oldSize, int newSize) {
```

```
    char *temp = realloc(buffer, newSize);

    if (temp == NULL) {
        printf("Buffer resizing failed!\n");

        /* Original buffer is still valid */
        free(buffer);
        exit(1);
    }

    printf("Buffer resized from %d to %d bytes.\n",
           oldSize, newSize);

    return temp;
}

/* ---------------- MAIN ---------------- */

int main() {

    int bufferSize = INITIAL_SIZE;
    char *buffer = createBuffer(bufferSize);

    printf("====================================\n");
    printf(" COMMAND HISTORY & DYNAMIC BUFFER\n");
    printf("====================================\n");

    printf("\nEscape Sequences:\n");
    printf("New Line      : \\n\n");
    printf("Tab           : \\tExample\n");
```

```
    printf("Type 'next' for next command.\n");
    printf("Type 'exit' to quit.\n\n");

    while (1) {

        printf("Command: ");

        /* Read input safely */
        if (fgets(buffer, bufferSize, stdin) == NULL)
            break;

        /* If input is too large, resize buffer */
        while (strchr(buffer, '\n') == NULL) {

            bufferSize *= 2;

            buffer = resizeBuffer(
                buffer,
                bufferSize / 2,
                bufferSize
            );

            int currentLength = strlen(buffer);

            if (fgets(buffer + currentLength,
                      bufferSize - currentLength,
                      stdin) == NULL)
                break;
```

```
      /* Remove newline */
      buffer[strcspn(buffer, "\n")] = '\0';

      /* Exit */
      if (strcmp(buffer, "exit") == 0)
          break;

      /* History */
      if (strcmp(buffer, "history") == 0) {
          showHistory();
          continue;
      }

      /* Previous command */
      if (strcmp(buffer, "prev") == 0) {
          previousCommand();
          continue;
      }

      /* Next command */
      if (strcmp(buffer, "next") == 0) {
          nextCommand();
          continue;
      }

      /* Store command */
      if (strlen(buffer) > 0) {
          addCommand(buffer);
          current = NULL;
          printf("Command stored: %s\n", buffer);
```

```
            continue;
        }

        /* Store command */
        if (strlen(buffer) > 0) {
            addCommand(buffer);
            current = NULL;
            printf("Command stored: %s\n", buffer);
        }
    }

    /* Release all allocated memory */
    free(buffer);
    freeHistory();

    printf("\nAll memory released successfully.\n");

    return 0;
}
himasree@LAPTOP-41COT5AA:~/OSLab$
```